

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Área Profesional Ingeniería

Introducción a la Programación y Computación



“Manual Técnico Proyecto 2 IPC2 2026 1 semestre”

César Enrique Ixquiac Vásquez./ Carné 202230332

Quetzaltenango, 12 de Marzo 2025

“Id y enseñad a todos”

1. Introducción y Objetivos

Objetivo: Detallar el diseño arquitectónico, la estructura del código en sus capas de Frontend y Backend, la lógica de transacciones financieras y los mecanismos de seguridad del sistema "ConnectWork". Este documento sirve como guía técnica para futuros mantenimientos, integración de nuevas pasarelas de pago y la escalabilidad del sistema.

Alcance del sistema: ConnectWork está diseñado bajo una arquitectura Cliente-Servidor orientada a servicios. Gestiona un mercado bilateral (*marketplace*) conectando Clientes y Freelancers. Implementa autenticación mediante JSON Web Tokens (JWT), persistencia de datos relacional para el control financiero (billeteras digitales y comisiones), generación de reportes analíticos y la gestión dinámica del catálogo de servicios.

2. Arquitectura del Sistema y Patrones de Diseño

A diferencia de las arquitecturas monolíticas de escritorio, este proyecto implementa una arquitectura web desacoplada:

- **Frontend (Single Page Application):** Desarrollado con el framework Angular (v17+). Utiliza una arquitectura basada en Componentes, Servicios observables (RxJS) y *Guards* para consumir recursos remotos de forma asíncrona, brindando una experiencia fluida.
- **Backend (API RESTful):** Desarrollado en Java Enterprise Edition (Servlets). Actúa como el motor lógico y financiero del sistema.
- **Patrón DAO (Data Access Object):** Separa estrictamente la lógica de acceso a datos de la lógica de negocio HTTP. Los Servlets nunca escriben código SQL; delegan esta tarea a las clases DAO.
- **Comunicación HTTP, JWT y CORS:** El intercambio de información entre el puerto 4200 (Angular) y el puerto 8080 (Tomcat) se realiza en formato JSON. Se implementó un sistema de Tokens (JWT) para mantener sesiones apátridas (stateless) seguras, junto con filtros CORS para autorizar peticiones de distintos orígenes.

3.1. Estructura Backend (Java)

Organizado modularmente en los siguientes paquetes:

- **Paquete `db`:** Contiene `ConexionDB.java`, aplicando el patrón Singleton para mantener un pool de conexiones optimizado hacia MySQL.
- **Paquete `modelo` (POJOs):** Clases Planas como `Usuario.java`, `Proyecto.java` y `Contrato.java` que estructuran la información para su serialización a JSON mediante `Gson`.
- **Paquete `dao`:**
 - `AuthDaoImpl.java`: Gestiona la verificación de contraseñas y la emisión de tokens JWT.
 - `ProyectoDaoImpl.java`: Maneja el ciclo de vida complejo de un proyecto (Publicación -> Propuestas -> Contratación -> Entrega -> Completado).

- **Paquete `controllers` (Servlets):** Punto de entrada del API. Capturan las peticiones HTTP (GET, POST), extraen los *Payloads* y retornan respuestas con sus respectivos códigos de estado (200 OK, 401 Unauthorized, 500 Internal Error).

3.2. Estructura Frontend (Angular)

- **Carpeta `services`:** Contiene servicios inyectables (ej. `auth.ts`, `proyecto.ts`) encargados de realizar las peticiones HTTP e interceptar/adjuntar el token JWT en las cabeceras (*Headers*).
- **Carpeta `components`:** Interfaz modularizada por roles. Destacan `dashboard-cliente.ts` y `dashboard-freelancer.ts`, que manejan el flujo de negocio mediante renderizado condicional (`@if`, `@for`) y modales de Bootstrap nativos.
- **Carpeta `guards`:** Contiene `auth-guard.ts`, que intercepta la navegación, decodifica el JWT (`jwt-decode`) y verifica tanto la caducidad de la sesión como el Rol de acceso antes de renderizar una ruta protegida.

4. Lógica de Negocio y Componentes Críticos

4.1. Transacciones Financieras y Billetera Virtual

El sistema maneja un entorno de "fondos en garantía". Cuando un Cliente acepta una Propuesta, el Backend inicia una transacción SQL con AutoCommit desactivado (`conn.setAutoCommit(false)`):

1. Verifica que el saldo del Cliente sea mayor o igual al monto de la propuesta.
2. Crea el registro del `contrato` calculando la comisión aplicable en ese instante.
3. Descuenta el dinero de la billetera del cliente (`cliente.saldo_disponible`). Si cualquier paso falla, se ejecuta un `conn.rollback()` para evitar inconsistencias de dinero.

4.2. Cierre de Contrato y Liberación de Fondos

Al aprobarse una `entrega`, ocurre otra transacción crítica: el dinero bloqueado en el contrato se divide en dos. Un porcentaje (la comisión) se inserta en `ingreso_plataforma` (ganancia del sistema), y el resto se suma al `saldo_acumulado` del Freelancer.

4.3. Motor de Reportes

Los reportes analíticos se procesan íntegramente a nivel de base de datos para optimizar recursos. Se utilizan cláusulas `BETWEEN` con la función `DATE()` para filtros temporales estrictos, `SUM()` y `GROUP BY` para obtener el gasto por categorías o top de ingresos de los trabajadores.

5. Modelo Relacional de Base de Datos

La base de datos (`connectwork`) implementa llaves foráneas en cascada y cardinalidad estricta:

- **Herencia a nivel de tablas:** Las tablas `cliente` y `freelancer` comparten una relación de 1:1 con la tabla principal `usuario`, heredando credenciales y datos demográficos pero separando métricas operativas (saldo).
- **Máquina de Estados (ENUM):** Tablas críticas como `proyecto` y `propuesta` tienen su ciclo de vida estrictamente definido por campos de tipo `ENUM` para evitar inconsistencias de software.

6. Requisitos y Entorno de Despliegue

Entorno Backend:

- **Lenguaje:** Java (JDK 17+)
- **Servidor de Aplicaciones:** Apache Tomcat 10.x.
- **Base de Datos:** MySQL Server 8.0+ (Motor InnoDB obligatorio).
- **Librerías Externas:** `mysql-connector-j.jar` (JDBC), `gson.jar` (JSON) y librerías compatibles con `java-jwt` para la emisión de tokens.

Entorno Frontend:

- **Plataforma:** Node.js v18+.
- **Framework:** Angular CLI v17+.

Pasos de Despliegue Local:

1. Ejecutar el script SQL en el puerto 3306.
2. Ajustar credenciales en `ConexionDB.java`.
3. Desplegar el archivo `.war` en Tomcat (Puerto 8080).
4. Desde la terminal del frontend, ejecutar `npm install` seguido de `ng serve`.
5. Acceder al sistema en `http://localhost:4200`.

Mapeo Físico DB (Esquema Relacional ConnectWork)

```
CREATE DATABASE IF NOT EXISTS connectwork;
```

```
USE connectwork;
```

```
CREATE TABLE categoria (  
    id_categoria INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(100) NOT NULL,  
    descripcion TEXT,  
    estado ENUM('ACTIVO', 'INACTIVO') DEFAULT 'ACTIVO'  
);
```

```
CREATE TABLE usuario (  
    id_usuario INT PRIMARY KEY AUTO_INCREMENT,  
    nombre_completo VARCHAR(100) NOT NULL,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    telefono VARCHAR(20),  
    direccion VARCHAR(200),  
    cui VARCHAR(13) NOT NULL UNIQUE,  
    fecha_nacimiento DATE NOT NULL,  
    rol ENUM('ADMIN', 'CLIENTE', 'FREELANCER') NOT NULL,  
    estado_cuenta ENUM('ACTIVO', 'INACTIVO') DEFAULT 'ACTIVO'  
);
```

```
CREATE TABLE cliente (  
    id_cliente INT PRIMARY KEY AUTO_INCREMENT,
```

```

    id_cliente INT PRIMARY KEY,

    descripcion_empresa TEXT,

    sector_industria VARCHAR(100),

    sitio_web VARCHAR(255),

    saldo_disponible DECIMAL(10,2) DEFAULT 0.00,

    FOREIGN KEY (id_cliente) REFERENCES usuario(id_usuario) ON DELETE
    CASCADE

);

CREATE TABLE freelancer (

    id_freelancer INT PRIMARY KEY,

    biografia TEXT,

    nivel_experiencia ENUM('JUNIOR', 'SEMI_SENIOR', 'SENIOR'),

    tarifa_hora DECIMAL(10,2),

    saldo_acumulado DECIMAL(10,2) DEFAULT 0.00,

    FOREIGN KEY (id_freelancer) REFERENCES usuario(id_usuario) ON DELETE
    CASCADE

);

CREATE TABLE habilidad (

    id_habilidad INT PRIMARY KEY AUTO_INCREMENT,

    id_categoria INT NOT NULL,

    nombre VARCHAR(100) NOT NULL,

    estado ENUM('ACTIVO', 'INACTIVO') DEFAULT 'ACTIVO',

    FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria) ON
    DELETE CASCADE

```

);

```
CREATE TABLE freelancer_habilidad (  
  
    id_freelancer INT,  
  
    id_habilidad INT,  
  
    PRIMARY KEY (id_freelancer, id_habilidad),  
  
    FOREIGN KEY (id_freelancer) REFERENCES freelancer(id_freelancer) ON  
DELETE CASCADE,  
  
    FOREIGN KEY (id_habilidad) REFERENCES habilidad(id_habilidad) ON  
DELETE CASCADE
```

);

```
CREATE TABLE proyecto (  
  
    id_proyecto INT PRIMARY KEY AUTO_INCREMENT,  
  
    id_cliente INT NOT NULL,  
  
    id_categoria INT NOT NULL,  
  
    titulo VARCHAR(200) NOT NULL,  
  
    descripcion TEXT NOT NULL,  
  
    presupuesto_maximo DECIMAL(10,2) NOT NULL,  
  
    fecha_limite DATE NOT NULL,  
  
    fecha_publicacion DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
    estado ENUM(  
  
        'ABIERTO',  
  
        'EN_REVISION',  
  
        'EN_PROGRESO',  
  
        'ENTREGA_PENDIENTE',
```

```

        'COMPLETADO',

        'CANCELADO'

    ) DEFAULT 'ABIERTO',

    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente) ON DELETE
    CASCADE,

    FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria)

);

CREATE TABLE proyecto_habilidad (

    id_proyecto INT,

    id_habilidad INT,

    PRIMARY KEY (id_proyecto, id_habilidad),

    FOREIGN KEY (id_proyecto) REFERENCES proyecto(id_proyecto) ON DELETE
    CASCADE,

    FOREIGN KEY (id_habilidad) REFERENCES habilidad(id_habilidad) ON
    DELETE CASCADE

);

CREATE TABLE propuesta (

    id_propuesta INT PRIMARY KEY AUTO_INCREMENT,

    id_proyecto INT NOT NULL,

    id_freelancer INT NOT NULL,

    monto_ofertado DECIMAL(10,2) NOT NULL,

    plazo_dias INT NOT NULL,

    carta_presentacion TEXT NOT NULL,

    estado ENUM(

        'PENDIENTE',

```



```

        'ACEPTADA',

        'RECHAZADA',

        'RETIRADA'

    ) DEFAULT 'PENDIENTE',

    fecha_envio DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (id_proyecto) REFERENCES proyecto(id_proyecto) ON DELETE
    CASCADE,

    FOREIGN KEY (id_freelancer) REFERENCES freelancer(id_freelancer) ON
    DELETE CASCADE

);

```

```

CREATE TABLE contrato (

    id_contrato INT PRIMARY KEY AUTO_INCREMENT,

    id_propuesta INT NOT NULL UNIQUE,

    monto_bloqueado DECIMAL(10,2) NOT NULL,

    porcentaje_comision_aplicado DECIMAL(5,2) NOT NULL,

    fecha_inicio DATETIME DEFAULT CURRENT_TIMESTAMP,

    motivo_cancelacion TEXT,

    FOREIGN KEY (id_propuesta) REFERENCES propuesta(id_propuesta) ON
    DELETE CASCADE

);

```

```

CREATE TABLE entrega (

    id_entrega INT PRIMARY KEY AUTO_INCREMENT,

    id_contrato INT NOT NULL,

    descripcion TEXT NOT NULL,

```

```
url_archivo VARCHAR(255),

estado ENUM('PENDIENTE','APROBADA','RECHAZADA') DEFAULT 'PENDIENTE',

motivo_rechazo TEXT,

fecha_subida DATETIME DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY (id_contrato) REFERENCES contrato(id_contrato) ON DELETE
CASCADE

);
```

```
CREATE TABLE calificacion (

    id_calificacion INT PRIMARY KEY AUTO_INCREMENT,

    id_contrato INT NOT NULL UNIQUE,

    estrellas INT CHECK (estrellas >= 1 AND estrellas <= 5),

    comentario TEXT,

    fecha_calificacion DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (id_contrato) REFERENCES contrato(id_contrato) ON DELETE
    CASCADE

);
```

```
CREATE TABLE historial_comision (

    id_historial_comision INT PRIMARY KEY AUTO_INCREMENT,

    porcentaje DECIMAL(5,2) NOT NULL,

    fecha_inicio DATETIME DEFAULT CURRENT_TIMESTAMP,

    fecha_fin DATETIME

);
```

```
CREATE TABLE ingreso_plataforma (

    id_ingreso INT PRIMARY KEY AUTO_INCREMENT,
```

```
id_contrato INT NOT NULL,  
  
monto_comision_cobrada DECIMAL(10,2) NOT NULL,  
  
fecha_ingreso DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
FOREIGN KEY (id_contrato) REFERENCES contrato(id_contrato) ON DELETE  
CASCADE  
);
```

```
CREATE TABLE recarga_saldo (  
  
id_recarga INT PRIMARY KEY AUTO_INCREMENT,  
  
id_cliente INT NOT NULL,  
  
monto DECIMAL(10,2) NOT NULL,  
  
fecha_recarga DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente) ON DELETE  
CASCADE  
);
```

```
CREATE TABLE solicitud_catalogo (  
  
id_solicitud INT PRIMARY KEY AUTO_INCREMENT,  
  
id_usuario_solicitante INT NOT NULL,  
  
tipo_solicitud ENUM('CATEGORIA','HABILIDAD') NOT NULL,  
  
nombre_sugerido VARCHAR(100) NOT NULL,  
  
descripcion TEXT,  
  
estado ENUM('PENDIENTE','ACEPTADA','RECHAZADA') DEFAULT 'PENDIENTE',  
  
FOREIGN KEY (id_usuario_solicitante) REFERENCES usuario(id_usuario) ON  
DELETE CASCADE  
);
```

